NEURAVIC

Backend System Architecture Design

Version 2.0 · 18 October 2026 · Status: proposal for review
Facts verified against repository neuravic-co/neuravic @ 2d9a1c2. Everything else is a proposal.

Summary (read this first)

• **What exists:** a finished marketing site (Next.js 16 on Cloudflare Workers). No API, database, forms or login. Contact is a mailto link; the Portal button is a placeholder.

• **What we build:** a modular-monolith API on Cloudflare Workers (Hono, D1, R2, KV, Queues) with modules for leads, auth, orgs, portal, files, content, admin, notifications and audit.

• **Ship first:** `POST /api/v1/leads` (section 8.1). It replaces the mailto link and every call-to-action uses it.

• **Why this shape:** three-person team, one language, one deployable, no servers; the frontend already compiles to a Worker.

• **Biggest risks:** custom authentication and D1 limits. Both are contained by design (section 13).

• **Decisions needed from you:** section 15. Five architecture decisions are written up as ADRs in Appendix A.

How to use this document

If you want to...	Go to
Start coding today	8.1 lead contract, 5.2 repo layout, 12 roadmap phase 0-1
Understand why we chose X	Appendix A (ADR-1 to ADR-5) and 14 trade-offs
Check capacity and failure plans	9 scale and reliability
Review security	10 security architecture

1. Context and current state

Neuravic is a multidisciplinary engineering company in Dubai and Cairo (AI systems, software platforms, product experience, deployment and operations). The public site presents **DARKAGENT** (governed multi-agent platform) and **COOLACCESS_DXB** (request, approval and access workflows). The backend serves the website and a planned client Portal; the two products are separate systems that connect to it.

Area	Verified today	Implication
Frontend	Next.js 16 / React 19 via vinext, Cloudflare Workers output, single page	Same-origin /api or a sibling Worker is possible
Data	db/schema.ts empty; getDb() Drizzle on D1 binding DB; hosting.json d1=null, r2=null	Create and bind D1, R2; first migration is ours
Contact	mailto:hello@neuravic.com	Needs capture, anti-spam, notification, storage
Portal	Header button shows toast only	No auth or portal service exists
Auth helper	app/chatgpt-auth.ts reads oai-authenticated-user-* headers, unused	Platform-specific; not for production
Libraries	zod, react-hook-form, drizzle-orm, ~60 shadcn/ui components	Shared validation and Portal UI kit ready
Quality	Only eslint; no tests or CI	Add CI, tests, migration gate

2. Requirements

2.1 Functional

• **FR1 Lead intake:** accept enquiries (name, email, company, message, service interest, budget), validate, store, notify Neuravic, acknowledge the sender.

• **FR2 Authentication:** passwordless login for Portal users, sessions, logout, admin invites.

• **FR3 Organisations and roles:** client orgs; owner, admin, member, viewer; plus internal staff roles.

• **FR4 Portal:** projects, service or change requests, approvals with comments, attachments, activity feed.

• **FR5 Admin:** lead inbox with status, notes, CSV export, user and org management.

• **FR6 Notifications:** email and Slack-compatible webhooks.

• **FR7 Content (optional):** services, projects and team as JSON.

• **FR8 Product bridge:** SSO so DARKAGENT and COOLACCESS_DXB trust Portal identity.

2.2 Non-functional

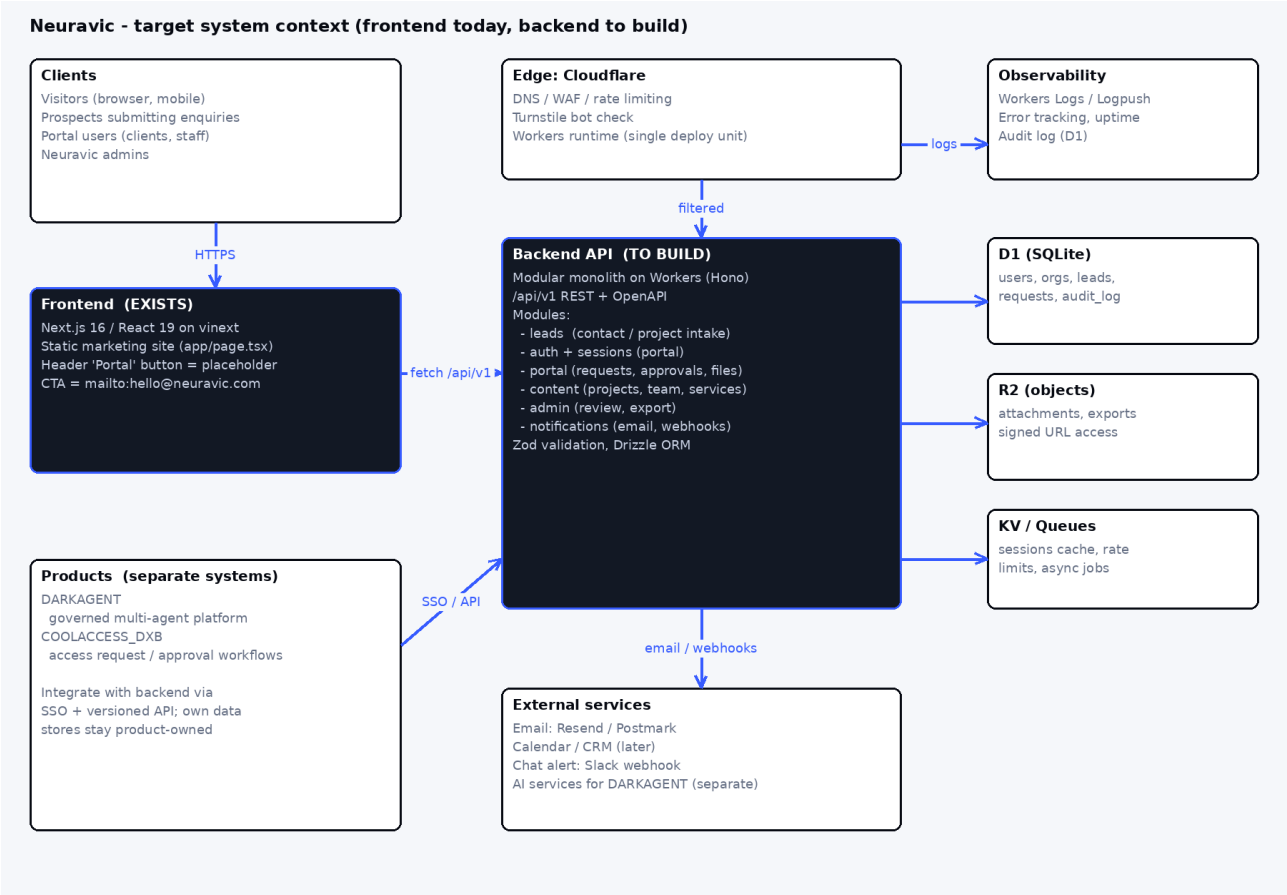
Attribute	Initial target
Availability	99.9% monthly for lead intake and login
Latency	p95 < 300 ms reads, < 600 ms writes at the edge
Security	OWASP ASVS L2 baseline, MFA-capable, least privilege
Privacy	Minimum personal data, delete/export, retention schedule; UAE and Egypt obligations to confirm with counsel (not legal advice)
Operability	One-command deploy and rollback, migrations in CI, structured logs, alerts
Cost	Near-zero idle cost; low hundreds of USD per month at 100x growth (estimate)

2.3 Constraints and assumptions

- **A1** Team: three people, one backend engineer, TypeScript throughout.
- **A2** Frontend stays on Cloudflare Workers via vinext.
- **A3** Portal users are client staff and Neuravic staff; both log in with email (to confirm, D2).
- **A4** DARKAGENT and COOLACCESS_DXB keep their own data stores.
- **A5** No regulatory requirement forces in-country hosting (to confirm, D6).

3. High-level design

Style: **modular monolith** on a Cloudflare Worker (Hono router) beside the existing frontend Worker, with managed storage. Calls go browser to Cloudflare edge (WAF, rate limit, Turnstile) to the API Worker to D1, R2, KV and Queues. Email and chat go out through the notification module.



3.1 Data flow: lead intake

```
Browser form -> POST /api/v1/leads (+Turnstile token)
  edge: rate limit per IP -> verify Turnstile -> zod validate -> honeypot/spam score
  leads.service: insert lead + outbox rows (one D1 batch/transaction)
  respond 202 {id}
  queue consumer: send team email, send sender acknowledgement, post Slack alert
  audit_log: lead.created
```

3.2 Data flow: portal login

```
POST /auth/magic-link (always 202) -> token stored hashed (15 min) -> email link
GET link -> POST /auth/verify -> session row + HttpOnly cookie
Subsequent calls: cookie -> session lookup (KV cache, D1 source) -> can(user,action,resource)
```

4. Component design

Module	Responsibility	Owns	Depends on
edge	WAF, rate limit, Turnstile, CORS, request id, security headers	KV counters	Cloudflare
leads	Create/list/update leads, spam score, dedupe	leads, lead_notes	notifications
auth	Magic link, passkeys, sessions, invites, token exchange	users, sessions, auth_tokens, passkeys	notifications, KV
orgs	Organisations, membership, RBAC checks	orgs, memberships	auth
portal	Projects, requests, approvals, comments, feed	projects, requests, approvals, comments	orgs, files
files	Signed upload/download, scan hook	files; R2	orgs
content	Services, projects, team	content_items	none
admin	Staff views and exports	none (via services)	all
notifications	Email, Slack, templates, retries	outbox, delivery_log	Queues, vendor
audit	Append-only security and data events	audit_log	all writers

Rule: modules call each other only via exported service interfaces, never another module's tables. Enforce with dependency-cruiser or eslint-plugin-boundaries.

4.1 Layering inside a module

```
route (Hono) -> zod validate -> authz policy -> service (logic, transaction)
  -> repository (Drizzle) -> D1 / R2 / KV
  -> domain event -> outbox -> Queue consumer (email, webhook, audit)
```

5. Repository layout and quick start

5.1 Proposed layout

```
neuravic/
  app/ components/ ...        existing frontend (unchanged)
  api/src/index.ts            Hono app, error handler, route mounting
  api/src/modules/{leads,auth,orgs,portal,files,content,admin,notifications,audit}/
  api/src/lib/{env,errors,ids,logger,rbac}.ts
  packages/shared/            zod schemas + types used by web and api
  db/schema.ts  drizzle/      migrations via drizzle-kit
  .github/workflows/ci.yml    lint, typecheck, test, migrate, deploy
```

5.2 First hour (phase 0)

```
npm i hono
wrangler d1 create neuravic-db          # then set d1 in .openai/hosting.json / wrangler config
# add tables to db/schema.ts, then:
npm run db:generate                      # existing script (drizzle-kit generate)
npm run dev                              # existing script; hit GET /api/v1/health
```

6. Data model

SQLite/D1 via Drizzle. IDs are ULIDs (text). Timestamps are UTC epoch integers. Soft delete (deleted_at) only where retention needs it.

Table	Key columns	Notes
users	id, email (unique, lowercase), name, status, staff_role?, created_at	Staff and clients together
orgs	id, name, slug, created_at	Client company
memberships	user_id, org_id, role	Composite PK
sessions	id (hash), user_id, expires_at, ip_hash, ua_hash	Server side, revocable
auth_tokens	id (hash), email, purpose, expires_at, used_at	Single use, 15 min
passkeys	id, user_id, public_key, counter, transports	WebAuthn
leads	id, name, email, company, message, service_interest, budget_band, source, status, spam_score, consent_at, created_at	Index (status, created_at), email
lead_notes	id, lead_id, author_id, body, created_at	
projects	id, org_id, name, status, created_at	Portal project
requests	id, project_id, type, title, body, state, requested_by, created_at	
approvals	id, request_id, approver_id, decision, comment, decided_at	
comments	id, request_id, author_id, body, created_at	
files	id, org_id, request_id?, r2_key, name, size, mime, sha256, scan_state, uploaded_by	Bytes in R2
content_items	id, kind, slug, data(json), published	Optional
outbox	id, topic, payload, state, attempts, next_attempt_at	Transactional outbox
audit_log	id, at, actor_id, action, target_type, target_id, ip_hash, meta	Append-only

Relations: orgs 1-N memberships N-1 users; orgs 1-N projects 1-N requests 1-N approvals, comments, files. Leads are standalone until converted.

Sketch: leads table in Drizzle

```
const LEAD_STATUSES = ['new','qualified','won','lost','spam'] as const;

export const leads = sqliteTable('leads', {
  id: text('id').primaryKey(),               // ULID
  name: text('name').notNull(),
  email: text('email').notNull(),
  message: text('message').notNull(),
  status: text('status', { enum: LEAD_STATUSES }).notNull().default('new'),
  spamScore: integer('spam_score').notNull().default(0),
  consentAt: integer('consent_at').notNull(),
  createdAt: integer('created_at').notNull(),
}, t => [
  index('leads_status_created').on(t.status, t.createdAt),
]);
```

7. Caching, queues and error handling

• **Caching:** public content endpoints use Cache-Control and edge cache; authenticated responses are never edge cached; session lookups cached in KV for 60 s.

• **Queues:** events written to the outbox in the same transaction, then drained to Cloudflare Queues; consumers retry with exponential backoff (1 m, 5 m, 30 m, 2 h, 12 h) then dead-letter.

• **Idempotency:** POST accepts Idempotency-Key; consumers are idempotent by outbox id.

• **Errors:** typed AppError mapped to RFC 9457 problem+json; unknown errors return a request id only; detail goes to logs.

• **Config:** typed env module validated at start-up with zod; separate local, preview, staging and production bindings.

8. API design

REST over HTTPS, JSON, base **/api/v1**, problem+json errors, cursor pagination (?cursor=&limit=), OpenAPI generated from zod.

Method + path	Auth	Purpose
POST /leads	public + Turnstile	Create lead (202)
GET /leads, GET/PATCH /leads/:id	staff	Inbox and status
POST /auth/magic-link	public, rate limited	Request login email (always 202)
POST /auth/verify; POST /auth/logout; GET /me	token / session	Session lifecycle
POST /auth/passkeys/*	session	WebAuthn
GET/POST /orgs/:id/members	org admin	Invite, manage
GET/POST /projects; GET /projects/:id	member	Portal projects
GET/POST /projects/:id/requests	member	Requests
POST /requests/:id/approvals; /comments	approver / member	Decide, discuss
POST /files/upload-url; GET /files/:id/download-url	member	Signed R2 URLs
GET /content/{services,projects,team}	public, cached	Site content
POST /token/exchange	session	Short JWT for products
GET /health, /version	public	Liveness, build info

8.1 Lead intake contract (ship first)

```
POST /api/v1/leads
{ "name": "1..120", "email": "email", "company": "string?",
  "message": "10..4000", "serviceInterest": ["ai"|"software"|"product"|"ops"],
  "budgetBand": "string?", "consent": true, "turnstileToken": "string",
  "website": ""   // honeypot, must be empty }

202 { "id": "01J...", "status": "received" }
400 validation   422 captcha failed   429 rate limited
```

Frontend change: replace the mailto button with a form built from the existing shadcn Form, Input and Textarea, react-hook-form and the shared zod schema; keep mailto as a fallback.

9. Scale and reliability

9.1 Load estimation (assumptions, revisit with real data)

Quantity	Estimate	Basis
Website visits	30,000 / month (about 1 req/min average)	Assumed small B2B site
Leads	50-300 / month (plus spam attempts up to 10,000)	Assumed 1% conversion; bots dominate
Portal users	50-500 active, peak 50 concurrent	A few dozen client orgs
Portal API calls	Under 5 requests/s peak	Workflow app, not consumer traffic
D1 size	Under 1 GB in year one	Text rows; files live in R2
R2 storage	10-100 GB in year one	Client attachments

Conclusion: load is far below Workers and D1 limits. Reliability and security, not throughput, drive the design.

9.2 Scaling and failure handling

• **Horizontal vs vertical:** Workers scale horizontally by platform. The limit to watch is D1 (single primary, size cap). Mitigation: indexes, read-heavy content cached at the edge, repository layer allowing a Postgres move (ADR-3).

• **Failover and redundancy:** Workers run multi-region; D1 has automatic replication of its primary storage. Third-party email is the weakest link: the outbox retries and a second provider can be added behind the same interface.

• **Backups:** D1 Time Travel for point-in-time restore plus a nightly export to a separate R2 bucket; restore drill every quarter. RPO 24 h or better, RTO 4 h.

• **Degradation:** if email fails, leads are still stored and retried. If Turnstile is down, fail closed on /leads with a clear message and mailto fallback.

• **Monitoring and alerting:** JSON logs (request id, hashed user id, module, latency) via Workers Logs/Logpush, uptime probe on /health, alerts on 5xx rate over 2%, queue backlog over 15 minutes, login failure spike, D1 size over 70% of cap.

10. Security architecture

• **Auth method:** passwordless. Magic link (single use, 15 min, hashed). Passkeys in phase 2. TOTP mandatory for staff.

• **Sessions:** opaque id in HttpOnly, Secure, SameSite=Lax cookie; hash server side; 30-day sliding, 12-hour absolute for staff.

• **CSRF:** SameSite plus double-submit token and strict Origin check.

• **Authorisation:** central can(user, action, resource) with per-org RBAC; deny by default; every route declares a policy.

• **Product SSO:** 5-minute EdDSA JWT (aud = product, claims sub, org, role), verified through a JWKS endpoint, key rotation via kid.

Threat	Control
Spam and bots on /leads	Turnstile, honeypot, per-IP and per-email rate limit, spam_score, quarantine state
Injection	Drizzle parameterised queries, zod on every input, encoded email templates
Account takeover	Short single-use tokens, no user enumeration, MFA for staff, session rotation, login alerts

Threat	Control
Broken access control	Deny-by-default policy, org scope in every query, per-role automated tests
Data exposure	Encryption at rest, short signed file URLs, hashed ip and user agent, no bodies in logs
Secrets	Wrangler secrets only, rotation runbook, secret scanning in CI
Supply chain	Lockfile, audit, Dependabot, pinned CI actions, few Worker dependencies
Upload abuse	Size and MIME allow list, sha256, scan hook, attachment disposition, separate bucket domain
Headers	CSP, HSTS, nosniff, Referrer-Policy, frame-ancestors none

Privacy: record consent_at, define retention (for example delete unqualified leads after 12 months), support delete and export. Legal basis and residency need counsel review.

11. Deployment and testing

Environment	Purpose	Data
local	wrangler dev --local --persist-to (as npm start does today)	Local SQLite, fixtures
preview	Per pull request	Ephemeral D1, migrations applied
staging	QA and demos	Anonymised only
production	neuravic.com with /api/* route	Real data, backups on

```
push -> CI: install, lint, typecheck, unit + integration (Miniflare), drizzle-kit check,
       build, OpenAPI diff, secret scan
main -> staging -> smoke tests -> manual gate -> migrations -> production
rollback: wrangler rollback (code); forward-only migrations (expand/contract)
```

• **Unit:** services and policy function (Vitest); 80% on auth, rbac, leads.

• **Integration:** Hono app on Miniflare D1/R2/KV, one test per endpoint and role.

• **Contract:** OpenAPI diff blocks breaking changes.

• **E2E:** Playwright for the lead form and login on preview.

• **Security:** dependency audit, authorisation matrix tests, external penetration test before opening the Portal to clients.

12. Delivery roadmap

Phase	Scope	Exit criteria
0 Foundations (1 wk)	api/ skeleton, bindings, CI, first migration, logging, /health	CI green, staging deploy
1 Lead intake (1-2 wk)	FR1, notifications, Turnstile, admin list endpoint, site form	Real enquiry stored and emailed, spam blocked
2 Auth and orgs (2-3 wk)	Magic link, sessions, orgs, roles, invites, audit	Role matrix tests pass; Portal button opens login
3 Portal core (3-5 wk)	Projects, requests, approvals, files, feed	Pilot client runs a real workflow
4 Admin and content (2 wk)	Console, exports, content API, retention jobs	Staff work without DB access
5 Product bridge (2 wk)	Token exchange, JWKS, SSO	One login across products

Estimates assume one backend engineer with frontend support. They are planning figures, not commitments.

13. Risks

Risk	Likelihood	Impact	Mitigation
Custom auth flaws	Medium	High	Vetted libraries or vendor, penetration test, staff MFA
D1 limits as Portal grows	Low-Med	Medium	Repository layer, indexes, Postgres path (ADR-3)
Scope creep into product features	Medium	Medium	Products own data; SSO and API only
Email deliverability	Medium	Medium	SPF, DKIM, DMARC; outbox retries
Regulatory or residency surprises	Low-Med	High	Counsel review in phase 0
Cloudflare lock-in	Medium	Low-Med	Standard HTTP and SQL, OpenAPI, hexagonal boundaries

14. Trade-off analysis

Decision	We gain	We give up
Modular monolith over microservices	One deploy, simple ops, easy refactors	Strict discipline to keep boundaries; shared failure domain
Cloudflare-native over Node + Postgres	No servers, low cost, matches frontend build	D1 size and feature limits; vendor coupling
Own auth over buying	Low cost, full control, product SSO fits	Security burden and maintenance
Outbox over fire-and-forget	No lost emails or events	More code and a drain job
Shared zod schema over separate DTOs	No form/API drift	Frontend and API versions must move together

What I would revisit as the system grows

• Move D1 to PostgreSQL if data passes about 5-10 GB, you need heavy reporting joins, or write contention appears.

• Buy auth (WorkOS or similar) when an enterprise client asks for SAML or SCIM.

• Split portal or notifications into a separate Worker when its release cadence or load diverges.

• Add a search index when lead or request lists exceed what indexed SQL filters handle.

• Revisit the 99.9% target and RPO/RTO once there are paying portal clients.

15. Decisions needed from you

ID	Decision	Recommendation
D1	Backend platform	Cloudflare Workers + Hono + D1/R2/KV; alternative Node + Postgres
D2	Portal audience	Both clients and staff, organised by org
D3	Auth build or buy	Build magic link + passkeys; buy if SAML is expected
D4	Email provider	Resend or Postmark on a neuravic.com subdomain
D5	Product integration depth	SSO only first
D6	Data residency and retention	Confirm with counsel before phase 2
D7	Repo layout	Monorepo as in 5.1

Appendix A. Architecture Decision Records

ADR-1: Run the backend on Cloudflare Workers with Hono

Status: Proposed **Date:** 18 October 2026 **Deciders:** Neuravic founders (backend owner + engineering lead)

Context. The frontend already compiles to a Cloudflare Worker and ships a D1 helper; the team is three people and wants no servers to operate.

Decision. Build the API with Hono on Cloudflare Workers, using D1, R2, KV and Queues.

Option	Complexity	Cost	Scalability	Team familiarity	Pros / cons
A: Workers + Hono	Low	Near zero idle	High (auto)	Medium	Fits existing build, no servers / D1 and runtime limits
B: Node (NestJS) + Postgres on a cloud VM or container	Medium	Fixed monthly	Medium	High	Mature ecosystem / servers, patching, extra latency to frontend
C: Managed BaaS (Supabase, Firebase)	Low	Per usage	Medium	Medium	Fast start / lock-in, custom workflow logic awkward

Trade-off analysis. A minimises operations and keeps the API next to the frontend. B is the safe fallback if D1 limits bite. C is fastest but fits poorly with approvals and product SSO.

Consequences.

• Easier: single deploy path, edge latency, low cost.

• Harder: Node-only libraries, long-running jobs, large queries.

• Revisit: if data or reporting needs outgrow D1 (ADR-3).

Action items.

• [ ] Create D1, R2, KV and Queue bindings

• [ ] Add api/ with Hono skeleton and /health

• [ ] Confirm decision D1

ADR-2: Use a modular monolith

Status: Proposed **Date:** 18 October 2026 **Deciders:** Neuravic founders (backend owner + engineering lead)

Context. Domains (leads, auth, portal, content, notifications) are small and tightly related; one engineer will build them.

Decision. Single deployable with enforced module boundaries and service interfaces between modules.

Option	Complexity	Cost	Scalability	Team familiarity	Pros / cons
A: Modular monolith	Low	Lowest	High enough	High	One deploy, easy refactor / needs boundary discipline
B: Microservices	High	Higher	Very high	Low	Independent scaling / ops overhead, distributed failures

Option	Complexity	Cost	Scalability	Team familiarity	Pros / cons
C: Unstructured monolith	Low	Lowest	Medium	High	Fastest start / hard to split later

Trade-off analysis. Microservices solve problems at a scale and team size we do not have. Boundaries keep the option open at low cost.

Consequences.

• Easier: transactions across modules, debugging, onboarding.

• Harder: keeping modules independent as pressure builds.

• Revisit: when a module needs its own release cadence or scaling.

Action items.

• [ ] Add dependency-cruiser rule for module imports

• [ ] Document module interfaces

ADR-3: Use D1 with Drizzle behind a repository layer

Status: Proposed **Date:** 18 October 2026 **Deciders:** Neuravic founders (backend owner + engineering lead)

Context. Drizzle and the D1 binding are already in the project. D1 is SQLite with size and single-primary limits.

Decision. Store relational data in D1 through Drizzle, with SQL confined to repositories.

Option	Complexity	Cost	Scalability	Team familiarity	Pros / cons
A: D1 + Drizzle	Low	Very low	Medium	High	Already wired, local-first / size cap, SQLite features
B: Postgres (Neon, Supabase) via Hyperdrive	Medium	Low-med	High	Medium	Richer SQL / extra vendor and latency hop
C: Durable Objects storage	High	Low	High per object	Low	Strong per-tenant isolation / unusual model

Trade-off analysis. A is sufficient for estimated volume (under 1 GB in year one). The repository layer makes B a contained migration.

Consequences.

• Easier: local dev, backups through Time Travel, zero ops.

• Harder: complex reporting and large exports.

• Revisit: at 5-10 GB, heavy joins, or write contention.

Action items.

• [ ] Write first migration with drizzle-kit

• [ ] Enforce 'no SQL outside repositories' lint rule

• [ ] Schedule nightly export to R2

ADR-4: Passwordless authentication with opaque server sessions

Status: Proposed **Date:** 18 October 2026 **Deciders:** Neuravic founders (backend owner + engineering lead)

Context. The Portal needs login for clients and staff. The existing chatgpt.site header helper is platform-specific. Passwords add reset, breach and reuse risk.

Decision. Magic link first, passkeys in phase 2, TOTP for staff, opaque revocable sessions, short JWT only for product SSO.

Option	Complexity	Cost	Scalability	Team familiarity	Pros / cons
A: Own passwordless + sessions	Medium	Low	High	Medium	Control, cheap, SSO fit / we own security
B: Vendor (Clerk, WorkOS, Auth0)	Low	Per user	High	Medium	Fast, SAML ready / cost, lock-in
C: Passwords + JWT	Medium	Low	High	High	Familiar / breach and reset burden, harder revocation

Trade-off analysis. A trades engineering time for control and cost; B trades money for reduced risk. Staying with A is reasonable while scope is small.

Consequences.

• Easier: instant revocation, no password handling, tailored product SSO.

• Harder: we own flows, rate limits and abuse handling.

• Revisit: on first SAML or SCIM request, or after a security review.

Action items.

• [ ] Threat-model auth flows

• [ ] Implement token hashing and rate limits

• [ ] Penetration test before client launch

ADR-5: Transactional outbox with Cloudflare Queues for side effects

Status: Proposed **Date:** 18 October 2026 **Deciders:** Neuravic founders (backend owner + engineering lead)

Context. Lead intake and workflow events must send email and webhooks reliably without slowing requests or losing messages.

Decision. Write events to an outbox table in the same transaction as the change, drain to Queues, make consumers idempotent.

Option	Complexity	Cost	Scalability	Team familiarity	Pros / cons
A: Outbox + Queues	Medium	Low	High	Medium	Reliable, retryable / more code
B: Inline calls in request	Low	Lowest	Low	High	Simple / slow, lost on failure
C: waitUntil fire-and-forget	Low	Lowest	Medium	High	Cheap / no retries or visibility

Trade-off analysis. A is the only option that guarantees delivery after partial failures.

Consequences.

• Easier: retries, dead letters, audit of deliveries.

• Harder: an extra table and a drain mechanism.

• Revisit: if volume demands a dedicated event bus.

Action items.

• [ ] Create outbox and delivery_log tables

• [ ] Build drain and consumer with backoff

• [ ] Alert on queue backlog